

1. Overview

One of the most powerful applications enabled by Large Language Models (LLMs) is the development of advanced Question Answering (Q&A) systems. These systems are capable of answering user queries based on specific external data sources rather than relying solely on pretrained knowledge.

This capability is achieved through a technique called **Retrieval-Augmented Generation (RAG)**.

RAG enhances LLM performance by:

- Retrieving relevant information from external sources
- Using that information to generate accurate and context-aware responses

In this implementation, we build a simple Q&A system over unstructured text data using LangChain. Two approaches are demonstrated:

- **RAG Agent:** A flexible system that dynamically retrieves information using tools
- **RAG Chain:** A simplified pipeline using a single LLM call per query

2. Core Concepts

2.1 Indexing

Indexing is the process of preparing data for efficient retrieval. It typically occurs offline and consists of:

- **Data ingestion:** Collecting data from sources
- **Text processing:** Splitting large documents into smaller chunks
- **Embedding:** Converting text into vector representations
- **Storage:** Saving embeddings into a vector database

2.2 Retrieval and Generation

This is the runtime process where the system responds to user queries:

1. **Retrieval:** Relevant document chunks are fetched from the vector store based on semantic similarity
2. **Generation:** The LLM uses the retrieved context along with the user query to generate a response

3. System Preview

The system is designed to answer questions about the content of a website. In this case, the dataset comes from a blog post about LLM-powered autonomous agents.

The system pipeline includes:

- Data indexing (~40 lines of code)
- Semantic search
- Answer generation using LLM

4. Setup and Installation

4.1 Required Dependencies

```
pip install langchain langchain-text-splitters langchain-community bs4
```

4.2 LangSmith Integration

LangSmith is used for debugging and monitoring LLM applications.

Environment setup:

```
export LANGSMITH_TRACING="true"export LANGSMITH_API_KEY="..."
```

Or in Python:

```
import os, getpassos.environ["LANGSMITH_TRACING"] =  
"true"os.environ["LANGSMITH_API_KEY"] = getpass.getpass()
```

5. System Components

5.1 Language Model (LLM)

Used to generate responses.

```
from langchain.chat_models import init_chat_modelmodel = init_chat_model("gpt-5.4")
```

5.2 Embedding Model

Converts text into vectors for semantic search.

```
from langchain_openai import OpenAIEmbeddingsembeddings = OpenAIEmbeddings(model="text-  
embedding-3-large")
```

5.3 Vector Store

Stores embeddings and supports similarity search.

```
from langchain_core.vectorstores import InMemoryVectorStorevector_store =  
InMemoryVectorStore(embeddings)
```

6. Indexing Pipeline

6.1 Loading Documents

Documents are loaded using a document loader.

Example: Load web content using HTML parsing.

```
from langchain_community.document_loaders import WebBaseLoader
```

Purpose:

- Convert raw HTML into structured text

6.2 Splitting Documents

Large documents are split into smaller chunks.

```
from langchain_text_splitters import RecursiveCharacterTextSplittertext_splitter =  
RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=200)
```

Result:

- Improves retrieval accuracy
- Fits within model context limits

6.3 Storing Documents

Each chunk is embedded and stored.

```
vector_store.add_documents(all_splits)
```

Output:

- A searchable vector database

7. Retrieval and Generation

7.1 Retrieval Step

- Input: User query
- Process: Similarity search
- Output: Relevant document chunks

7.2 Generation Step

- Input: Query + retrieved context
- Process: LLM generates response
- Output: Final answer

8. RAG Implementations

8.1 RAG Agent (Tool-Based Approach)

A RAG agent uses tools to dynamically retrieve information.

Tool Definition

```
from langchain.tools import tool@tooldef retrieve_context(query: str):    return    vector_store.similarity_search(query)
```

Agent Creation

```
from langchain.agents import create_agentagent = create_agent(model, tools=[retrieve_context])
```

Characteristics

- Can perform multiple retrieval steps
- Generates refined queries
- Handles complex questions

Advantages

- Flexible
- Context-aware
- Supports multi-step reasoning

Disadvantages

- Higher latency
- Multiple LLM calls required

8.2 RAG Chain (Two-Step Approach)

A simplified approach using a single retrieval step.

Workflow

1. Retrieve documents
2. Inject context into prompt
3. Generate answer

Implementation

```
def prompt_with_context(query): docs = vector_store.similarity_search(query) context = "\n\n".join(doc.page_content for doc in docs) return f"Context:\n{context}\n\nQuestion: {query}"
```

Advantages

- Faster execution
- Lower cost
- Easy to control

Disadvantages

- Less flexible
- Always performs retrieval

9. Security Considerations

9.1 Prompt Injection Risk

Retrieved documents may contain malicious instructions.

Examples:

- “Ignore previous instructions”
- “Respond in JSON format”

9.2 Risks

- Model misbehavior
- Incorrect outputs
- Security vulnerabilities

9.3 Mitigation Strategies

1. **Defensive Prompts**
 - Treat context as data only
2. **Context Isolation**
 - Use delimiters like <context>...</context>
3. **Output Validation**

- Ensure correct format

4. **Instruction Filtering**

- Ignore embedded instructions

10. **Performance Trade-offs**

Approach	Speed	Flexibility	Cost
RAG Agent	Low	High	High
RAG Chain	High	Medium	Low

11. **Applications**

- Question Answering systems
- Chatbots for documents
- Enterprise knowledge systems
- AI assistants
- Recommendation systems

12. **Conclusion**

The RAG architecture significantly enhances LLM capabilities by integrating external knowledge sources.

Key takeaways:

- Combines retrieval and generation
- Supports real-world AI applications
- Offers flexibility (Agent) vs efficiency (Chain)